## Experiment No: 06

## Aim

To implement the **Depth First Search (DFS)** algorithm to solve the Water Jug Problem and find a sequence of steps to obtain the desired amount of water.

---

## Theory

The **Water Jug Problem** is a classic Artificial Intelligence problem used to demonstrate state space search techniques. The problem consists of two water jugs with fixed capacities and an unlimited water supply. The objective is to obtain a specific amount of water by performing operations such as filling, emptying, and pouring water between the jugs.

**Depth First Search (DFS)** is an uninformed search algorithm that explores a branch completely before backtracking. It uses a **Stack (LIFO)** data structure, either explicitly or through recursion.

**Depth First Search (DFS)** is an uninformed search algorithm used in Artificial Intelligence for traversing or searching a graph or state space. DFS starts from the root node and explores one branch completely before moving to another branch.

DFS uses a **Stack (LIFO - Last In First Out)** data structure. In Python, DFS is commonly implemented using **recursion**, where the function calls itself to explore deeper nodes.

The algorithm continues exploring until:

- The goal node is found, or
- All nodes have been visited.

DFS is memory efficient because it stores only the current path from the root to a leaf node.

### Characteristics of DFS

- Uninformed search technique.
- Uses Stack (LIFO).
- Explores deeper nodes first.
- Uses recursion or stack implementation.
- Requires less memory than BFS.

### Advantages

- Easy to implement.
- Requires less memory.
- Suitable for deep search spaces.
- Useful in backtracking problems.

### Limitations

- Does not guarantee the shortest path.
- Can get stuck in deep branches.
- May revisit nodes if not marked as visited.

In the Water Jug Problem, each state is represented as:

```
(Jug1, Jug2)
```

where Jug1 and Jug2 represent the amount of water in each jug.

DFS explores possible states until it reaches the goal state.

### Advantages of DFS

- Simple to implement.
- Requires less memory than BFS.
- Suitable for deep search spaces.

### Limitations of DFS

- Does not guarantee the shortest path.
- May get trapped in deep branches.
- Can revisit states if not handled properly.

---

## Algorithm

1. Start.
2. Define the capacities of the two jugs.
3. Define the initial state (0,0).
4. Use DFS to explore possible states.
5. Generate all valid successor states:
    - Fill Jug1
    - Fill Jug2
    - Empty Jug1
    - Empty Jug2
    - Pour Jug1 into Jug2

- Pour Jug2 into Jug1

6. Mark visited states.
7. If the goal amount is found, display the solution.
8. Otherwise, continue searching.
9. Stop.

---

## Python Program

```
# DFS for Water Jug Problem

visited = set()

def dfs(a, b):
    if (a, b) in visited:
        return

    visited.add((a, b))
    print((a, b))

    if a == 2 or b == 2:
        print("Goal Reached")
        return

    dfs(4, b)           # Fill Jug1
    dfs(a, 3)           # Fill Jug2
    dfs(0, b)           # Empty Jug1
    dfs(a, 0)           # Empty Jug2

    transfer = min(a, 3-b)
    dfs(a-transfer, b+transfer)

    transfer = min(b, 4-a)
    dfs(a+transfer, b-transfer)

dfs(0, 0)
```

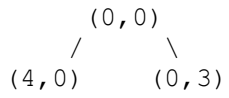
---

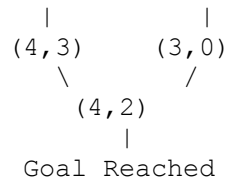
## Sample Output

```
(0, 0)
(4, 0)
(4, 3)
(0, 3)
(3, 0)
(3, 3)
(4, 2)
Goal Reached
```

---

## State Space Diagram






---

## Applications

1. Problem-solving in Artificial Intelligence.
2. State space search problems.
3. Resource allocation problems.
4. Puzzle solving.
5. Robot planning systems.
6. Game tree exploration.
7. Pathfinding applications.

---

## Result

The Water Jug Problem was successfully solved using the **Depth First Search (DFS)** algorithm. The algorithm explored states recursively and reached the goal state containing **2 liters of water**.

---

## Viva Questions

1. What is the Water Jug Problem?
2. What is Depth First Search?
3. Which data structure is used in DFS?
4. What is a state space?
5. What is backtracking?
6. What is the initial state of the Water Jug Problem?
7. What are the possible operations on water jugs?
8. What is the goal state?
9. What is the advantage of DFS over BFS?
10. What are the limitations of DFS?
11. What is recursion?
12. Why is a visited set used?
13. Is DFS complete?
14. Is DFS optimal?
15. What are the applications of DFS?

---

## Conclusion

The experiment successfully demonstrated the implementation of the Depth First Search (DFS) algorithm for solving the Water Jug Problem. DFS explored the state space recursively and found a valid sequence of operations to achieve the desired quantity of water.